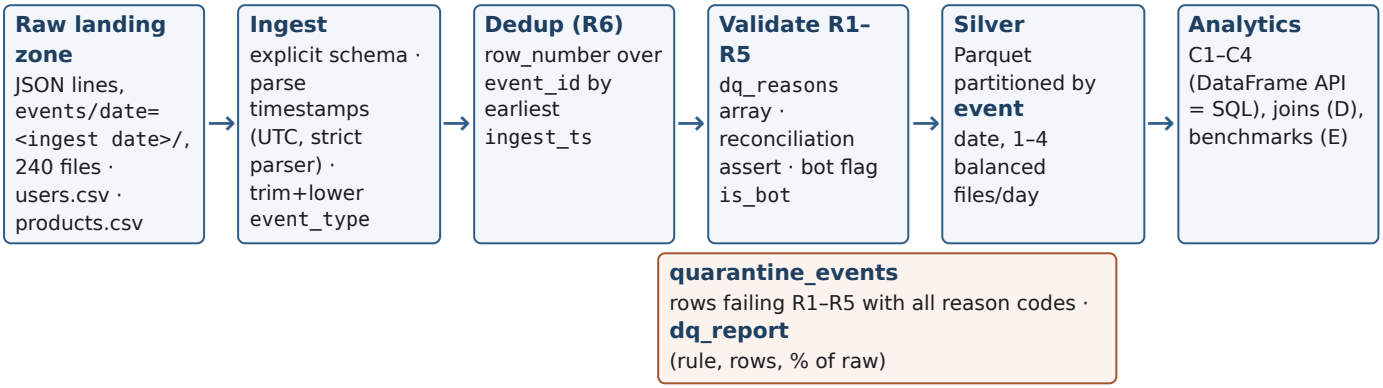
ShopStream at Scale — Distributed Batch Analytics with PySpark

Dana Ibaidullina · MSc Applied Artificial Intelligence, Astana IT University · Midterm assignment A2 · dataset **--scale medium --seed 5820** (2,828,711 raw events, 60 days, 30,000 users, 5,000 products)

Environment. Windows 11 (10.0.26200), AMD Zen 5 laptop CPU (Family 26 Model 96), 16 logical cores, 16.2 GB RAM · OpenJDK 17.0.18 (Zulu) · Apache Spark 3.5.3 / PySpark, `local[*]`, driver memory 8 GB · Python 3.11.17. **Method.** Every time is the median of 3 runs after one untimed warm-up run; `spark.catalog.clearCache()` between runs; execution forced with `write.format("noop")`. **Reproduce:** `python run_a2.py --scale medium --seed 5820` executes the notebook top-to-bottom and writes all CSV/PNG to `results/`.

1. Pipeline



2. Part A — Ingestion and storage layout

A1. Schema inference vs explicit schema

Variant	read only, s	read + count(), s
Schema inference	1.311	1.586
Explicit schema	0.127	0.567
Ratio	10.3×	2.8×

JSON has no embedded schema, so inference triggers an **extra full scan** of all 240 files before any query runs — visible in "read only" (1.31 s spent before `count()`). The explicit schema needs one pass and gives stable types: e.g. `event_ts` is kept as string and parsed by us, so malformed values become R2 violations instead of breaking the read.

A3. Pruning & pushdown — "electronics revenue, 10–16 Aug"

Storage	median, s	revenue
Raw JSON	1.541	428,810.69
Parquet, unpartitioned	0.436	424,863.19
Parquet, partitioned by event_date	**0.305**	424,863.19

Partitioned Parquet is 5.1× faster than JSON and 1.4× faster than flat Parquet. Only the partitioned plan has `PartitionFilters` on `event_date` (7 of 60 folders opened); the flat file relies on `PushedFilters` (row-group min/max skipping); JSON pushes only `IsNotNull(product_id)` and parses every line. JSON revenue is 0.93% higher because duplicates and invalid rows are not removed — an argument for the silver layer. Annotated plans: Appendix A.

A2. Silver write strategies (2,764,395 rows, 60 day-partitions)

Strategy	files	files/day	min KB	median KB	max KB	total MB	write, s
(i) direct `partitionBy("event_date")`	960	16	53.8	150.1	215.3	145.65	7.24
(ii) `repartition("event_date")`	60	1	1,617.8	1,762.1	2,348.2	109.68	**3.70**
(iii) balanced: day gets [4·rows/max_day_rows] buckets by `xxhash64(event_id)`, `repartitionByRange(P, part_id)`	198	3.3	548.7	666.0	714.3	128.46	8.56

Small-files problem in numbers. In (i) each of the 16 read tasks holds rows of every day, so it writes one file into every folder: 16 × 60 = 960 files of ~150 KB. The same data then takes **+33% disk** (145.7 vs 109.7 MB: per-file footer/metadata and worse compression of small row groups), and every later scan must list and open 16× more files. (ii) is fastest here because a day is only ~1.8 MB, but it sends a whole day to one task — at 100× that becomes a single multi-GB task. (iii) keeps files within 1.3× of each other (549–714 KB) and scales the file count with day size; it is slower to write because `repartitionByRange` samples the data first and needs an extra join. We use (iii) as the silver table.

3. Part B — Data quality and cleaning

Rule	Definition	rows	% raw
R1	user_id is NULL	22,243	0.786
R2	event_ts unparseable, < 2026-01-01 or > ingest_ts	5,643	0.199
R3	price ≤ 0 or NULL	8,535	0.302
R4	event_type invalid after trim+lower	0	0.000
R5	product_id not in products.csv	11,300	0.399
R6	duplicate event_id (later copies)	16,871	0.596
quarantine	**≥ 1 of R1–R5 (276 rows ≥ 2)**	**47,445**	**1.677**
silver	**valid rows**	**2,764,395**	**97.726**

Reconciliation (assert): 2,828,711 = 2,764,395 + 47,445 + 16,871 ✓.

Dedup before validation. A duplicate is the same event delivered twice (same payload, later `ingest_ts`). Removing copies first means each event is validated exactly once, so a bad event is quarantined once and the rule counts in dq_report are not inflated; it also makes the reconciliation terms disjoint. We keep the earliest copy via `row_number()` (`dropDuplicates` keeps an arbitrary one).

R4 = 0 by design: 28,402 messy values (`" VIEW "`, `"Purchase"`) are *repaired* by normalisation instead of being discarded. Late events: only 26 silver events have an ingest date different from their event date; partitioning by `event_date` puts them in the correct day.

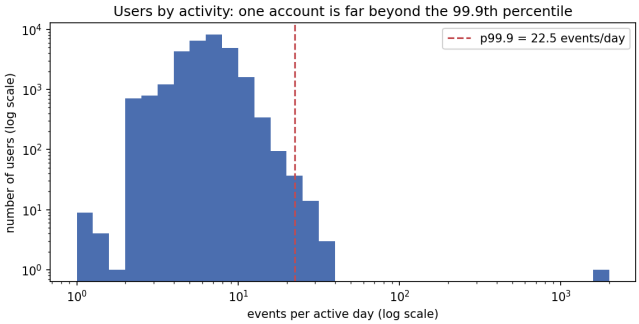


Fig. 1 — Events per active day per user (log–log). One account lies ~80× beyond the 99.9th percentile.

Bot detection (no hard-coding). A user is flagged when `events_per_day > p99.9` (22.5) **and** `n_events > p99.9` (1,044) **and** there is no cart/purchase activity. Both thresholds are computed from the data with `percentile_approx`. The second condition was added after we found that the first rule alone also flagged a one-day human user with 23 views. Result: **1 user, 106,312 events (3.85% of silver)**, ~1,772 events/day for 60 days vs a median user of 6.5 — flagged via `is_bot`, not deleted.

4. Part C — Analytics (bots excluded, 2,658,083 events)

Every query is implemented twice (DataFrame API and Spark SQL) and checked with `exceptAll` in both directions (doubles rounded to 6 decimals to ignore summation order): C1 180 rows, C2 2,658,083 rows, C3 450 rows, C4 60 rows — all identical.

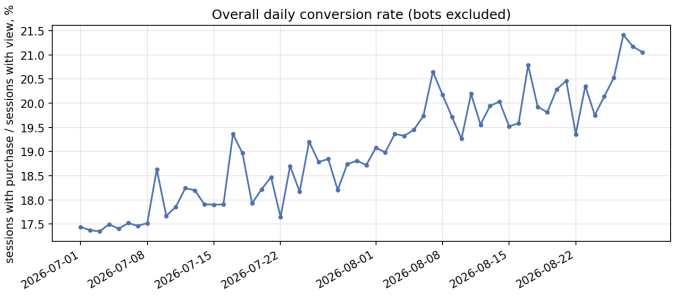


Fig. 2 — Daily conversion: sessions with a purchase / sessions with a view.

C1 funnel (event_date × device; sessions with ≥1 view / cart / purchase, two step rates, GMV = Σ price·qty of purchases). View→cart ≈ 0.38–0.47 and cart→purchase ≈ 0.40–0.54 on all devices; overall conversion rises from 17.4% to 21.4% while DAU falls (churn removes low-propensity users first). Note: purchases can occur without a cart event in this data, so cart→purchase is a ratio of session counts, not a strict funnel.

C2 sessionization (`lag` + running `sum` of "gap > 30 min" flags per user): **444,399** rebuilt vs 454,582 original sessions (−2.2%); median events 4 → 5, median duration 158 → 163 s; **0%** of original sessions split, **4.37% merged**. Example: user `u_000014` on 30 Jul has session `s_00000087` (14:00–14:09) and `s_00000086` starting 14:23:47 — a 14.6-min pause, so both become `u_000014_12`. Within an original session gaps average ~45 s, so splits never occur.

C3 top products (`dense_rank` by weekly revenue per category, share of category revenue): demand is very concentrated — the #1 product holds ~62–64% of *auto*, ~55% of *beauty* and ~50% of *fashion* revenue every week, but only ~5% in *books*. ISO weeks 27 and 35 are partial (data starts Wed 1 Jul, ends Sat 29 Aug). **C4 stickiness.** DAU/WAU without `countDistinct` over a sliding window: each distinct (user, active_day) is exploded with `sequence(d, d+6)` into the 7 days whose WAU it counts toward, then distinct users are counted per day. Brute force on 3 days (01 Jul: 5,906; 31 Jul: 18,675; 29 Aug: 15,441) matches exactly. DAU/WAU ≈ 0.28–0.32 on weekdays vs 0.36–0.40 at weekends; WAU declines from 21,135 to 15,441 (churn). The first 6 days have an incomplete window.

5. Part D — Joins, skew and AQE (bots kept; auto-broadcast disabled for D1–D3)

Diagnosis. Top key `u_000000`: 106,312 rows = **1,772× the median user** (60 rows); the next key has 1,805. After hashing `user_id` into 200 partitions: max 119,530, median 13,202.5, min 8,679 rows → **max/median = 9.1×** (Fig. 3). Hot keys are detected programmatically as keys larger than an average partition (13,822 rows) → `['u_000000']`.

Strategy (median of 3)	s	plan
(a) broadcast hash join	**0.67**	BroadcastHashJoin
(b) sort-merge, AQE off	5.13	SortMergeJoin
(c) sort-merge, AQE skew join	6.88	SortMergeJoin(skew=true)
(d) salting N = 4, AQE off	5.16	SMJ on (user_id, salt)
(d) salting N = 16	5.11	″
(d) salting N = 64	5.20	″

All six produce **2,764,395 rows and the same order-independent checksum** (Σ xxhash64 over all 18 output columns), which proves the salted join equals the unsalted one.

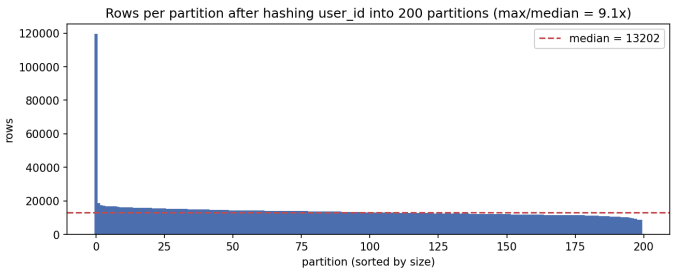


Fig. 3 — Rows per partition after hashing user_id into 200 partitions.

AQE tuning. With the default threshold (256 MB) skew handling cannot trigger: the bot partition is orders of magnitude smaller (the largest join partition in the Spark UI, Fig. 6, is 131 KiB of compressed shuffle data). The notebook lowers `skewedPartitionThresholdInBytes` step-wise and keeps the first value whose *final* plan (`isFinalPlan=true`) contains `SortMergeJoin(skew=true)` and `AQEShuffleRead: coalesced and skewed`: factor 3, threshold 1 MB (Appendix B).

Salting adds `salt = pmod(xxhash64(event_id), N)` only for hot keys (0 otherwise) and replicates the hot user's dimension row N times with `explode(sequence(0, N-1))`; the plan shows `hashpartitioning(user_id, salt, 200)`.

Interpretation. At this scale **broadcast wins by 7.7×**: users.csv (30k rows) fits in every task, so the 2.8M-row fact table is never shuffled and skew is irrelevant. Skew mitigations do not pay off locally: AQE's split (with a 1 MB threshold it also issues many tiny tasks and re-reads the build side per split) is 34% slower than plain SMJ, and salting is neutral at any N. **Local mode understates the skew penalty** because (1) 16 cores run 200 tasks in ~13 waves, so the one long task overlaps with the others instead of determining the stage end; (2) shuffle data stays on the local SSD — no network transfer of the hot partition to one executor; (3) the hot partition (~20 MB) fits in memory, so there is no spill or GC pressure; (4) tasks take tens of ms, so fixed scheduling cost dominates. The Spark UI confirms it (Fig. 6): the slowest task takes 300 ms vs a 73 ms median, but with 16 cores it simply overlaps with ~13 waves of other tasks. **On a 50-node cluster** (≈200 cores) all 200 partitions run in one wave, so the join stage lasts as long as the largest task: ≈ 9.1× the balanced time, with 199 cores idle. At 100× the bot partition is ~2 GB and would spill; there AQE skew join with default thresholds (or salting) becomes necessary unless the dimension is broadcast or bots are filtered before the join.

6. Part E — Partitioning, caching and scaling (W = C1 + C3 + C4)

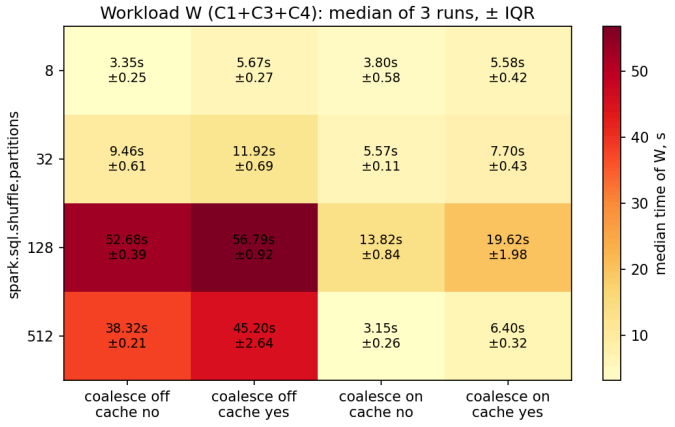


Fig. 4 — W time: shuffle.partitions × AQE coalescing × cache (median ± IQR, 3 runs).

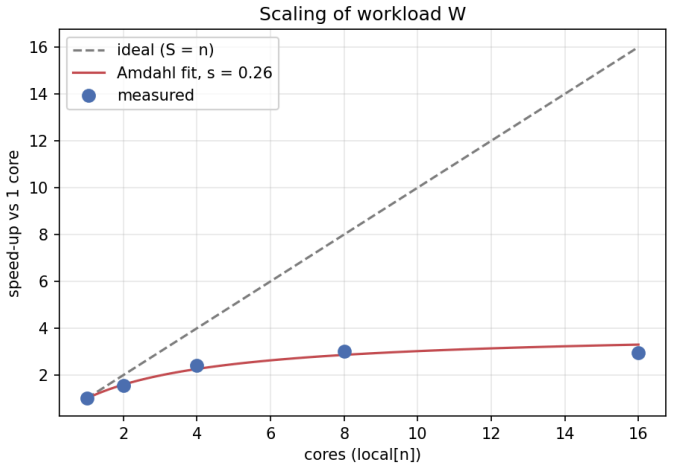


Fig. 5 — Speed-up vs cores (new SparkSession per point), Amdahl fit.

Grid. Best: 512 partitions + AQE coalescing, no cache — **3.15 s**; worst: 128 partitions, no coalescing, cache — 56.79 s (18×). Without coalescing the time is **non-monotonic** (8 → 128 partitions: 3.4 → 52.7 s, then 512: 38.3 s). Hypothesis: the bypass-merge shuffle writer, used for ≤ 200 partitions, makes every map task open one file per reduce partition; on Windows file creation (incl. antivirus scanning) is expensive. Test (E1b) with `bypassMergeThreshold=1`:

partitions (no coalescing)	bypass on, s	bypass off, s	speed-up
32	9.46	3.57	2.6×
128	52.68	8.57	6.1×

Confirmed: shuffle-file count, not computation, dominated. AQE coalescing removes most of the penalty automatically (128: 52.7 → 13.8 s). **Caching was always slower** (e.g. 3.15 → 6.40 s): building the in-memory columnar cache costs more than re-reading ~110 MB of Parquet already in the OS page cache.

master	cores	median s	IQR	speed-up	Amdahl
local[1]	1	8.57	0.13	1.00	1.00
local[2]	2	5.46	0.16	1.57	1.59
local[4]	4	3.55	0.02	2.41	2.26
local[8]	8	2.83	0.09	3.03	2.86
local[*]	16	2.91	0.08	2.95	3.30

Amdahl: least-squares fit of $1/S - 1/n = s(1 - 1/n)$ gives **$s = 0.257$** (max speed-up $1/s \approx 3.9\times$); on *small* it was $s = 0.50$, i.e. the parallel share grows with data size. **Serial part:** single driver (planning 3 queries, scheduling dozens of stages, Py4J calls), listing 60 folders/198 files, single-partition final aggregations and global windows, per-stage fixed costs; plus shared disk/memory bandwidth and SMT — 16 logical cores are likely 8 physical, so 16 is no faster than 8.

7. Three findings for the platform lead

- **Shuffle-file count is our bottleneck, not CPU.** With 128 partitions and no AQE coalescing, W takes 52.7 s; disabling the bypass writer gives 8.6 s (6.1×) and AQE coalescing 13.8 s. Keep `spark.sql.adaptive.coalescePartitions.enabled=true` with a generous initial partition count (best: 512 + coalescing, 3.15 s), and never benchmark on one OS and deploy on another without re-measuring.
- **Layout pays every day.** Explicit schema reads 10× faster than inference; partitioned Parquet answers the weekly revenue question 5.1× faster than JSON; a naive write produces 16 files/day and +33% storage. Write silver with a size-aware file strategy and an explicit schema contract.
- **One bot distorts both metrics and performance.** 1 account = 3.85% of events, 1,772× the median user, 9.1× partition skew. Flag it in silver, exclude it from KPIs, and broadcast small dimensions (7.7× faster than sort-merge); keep AQE skew join ready for when the dimension no longer fits in memory.

8. What changes at 100× (~70 GB raw JSON/day)?

Scaling our measurements: ~283M events/day; silver Parquet ≈ 6.4× smaller than JSON → ≈ **11 GB/day**; shuffle footprint measured at ≈ 190 B/row → ≈ **54 GB** per full shuffle of a day's events.

- **Files:** target **256 MB** Parquet files → ≈ **44 files per event_date partition** (strategy iii with buckets = ⌈day_bytes / 256 MB⌉), daily compaction of late data; no sub-hour partitioning (would create small files again).
- **Shuffle partitions:** start at **~400** (54 GB / ~128 MB) with AQE on, `advisoryPartitionSizeInBytes=128MB`, coalescing on; at 400 > 200 the sort-based shuffle writer is used, avoiding the file explosion seen in E1.
- **Executors:** e.g. 20 executors × 5 cores × **16 GB** heap (+2–3 GB overhead), ≈ 3 GB per core for 128–200 MB partitions with room for sorts; dynamic allocation 10–40 executors.
- **Joins:** users grows to ~3M rows (~150 MB CSV, ~0.3–0.5 GB in memory): broadcast still feasible with `autoBroadcastJoinThreshold≈512MB` and 16 GB executors. If not, sort-merge with AQE skew join at **default** thresholds: the bot partition (~3.85% × 54 GB ≈ 2 GB) exceeds both 256 MB and 5× the median (~135 MB), so it is split automatically; salting N≈16 for known heavy keys. Filter `is_bot` before analytic joins.
- **Caching:** only for data reused many times after expensive steps (e.g. deduplicated day before C1–C4), with `MEMORY_AND_DISK`; the E1 result shows caching cheap Parquet scans is a net loss.

9. AI-use statement

I used **Claude (Anthropic)** as a tutor and pair-programmer: to explain Spark concepts (shuffle, AQE, pruning, salting, Amdahl's law), to propose code for individual notebook cells which I ran, debugged and adapted on my machine, to diagnose Windows setup issues (Java, winutils), and to draft the wording of this report from my own measured results. All numbers come from my runs with seed 5820; I can explain and modify every line of the submitted code.

Appendix

A. Query plans for A3 (key lines of `explain("formatted")`, medium)

```
[1] Raw JSON
Scan json  Location: .../shopstream_medium_5820/events
PushedFilters: [IsNotNull(product_id)]                          ← only a null check; nothing to prune
ReadSchema: struct<product_id:string,event_type:string,event_ts:string,price:double,quantity:bigint>
Filter: lower(trim(event_type)) = purchase AND gettimestamp(event_ts, ...) >= 2026-08-10 ...
                                                                ← functions on columns cannot be pushed; every line is
parsed

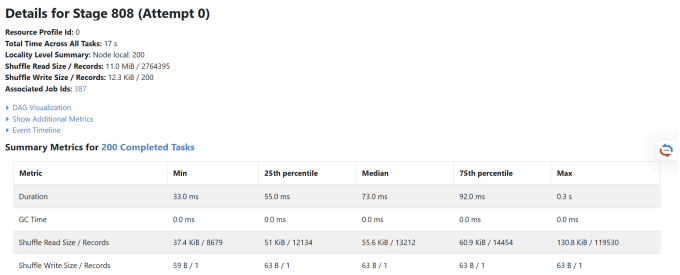
[2] Parquet, unpartitioned (silver_flat)
PushedFilters: [IsNotNull(event_type), IsNotNull(event_date), EqualTo(event_type,purchase),
                GreaterThanOrEqual(event_date,2026-08-10), LessThanOrEqual(event_date,2026-08-16), IsNotNull(product_id)]
                                                                ← pushed to the Parquet reader: row groups skipped by
min/max stats
ReadSchema: struct<product_id:string,event_type:string,price:double,quantity:bigint,event_date:date>
                                                                ← column pruning: 5 of 13 columns read; no PartitionFilters
→ all files opened

[3] Parquet, partitioned by event_date (silver_iii_balanced)
PartitionFilters: [isnotnull(event_date), (event_date >= 2026-08-10), (event_date <= 2026-08-16)]
                                                                ← partition pruning: only 7 of 60 folders listed and read
PushedFilters: [IsNotNull(event_type), EqualTo(event_type,purchase), IsNotNull(product_id)]
ReadSchema: struct<product_id:string,event_type:string,price:double,quantity:bigint>
                                                                ← event_date not read from files: its value comes from the
folder name
All three: BroadcastHashJoin with products filtered by PushedFilters [EqualTo(category,electronics)].
```

B. Final AQE plan for D2 (c), proving the skew join was applied

```
AdaptiveSparkPlan  isFinalPlan=true
+- == Final Plan ==
   * SortMergeJoin(skew=true) Inner                             ← join executed in skew mode
     :- AQEShuffleRead  Arguments: coalesced and skewed         ← events side: bot partition split, small ones merged
     +- AQEShuffleRead  Arguments: coalesced                     ← users side: only coalesced (matching rows replicated per
split)
+- == Initial Plan ==
   SortMergeJoin Inner                                          ← before runtime statistics: an ordinary SMJ
Settings: skewJoin.enabled=true, skewedPartitionFactor=3, skewedPartitionThresholdInBytes=1024KB,
          advisoryPartitionSizeInBytesInBytes=256KB (calibration: results/D2_aqe_skew_calibration.csv)
Salted plan (d): Exchange hashpartitioning(user_id, salt, 200); users side = Union(Generate explode(sequence(0,N-1)) for
hot keys,
                 Project 0 AS salt for the rest)
```

C. Spark UI — join stage task summary



Details for Stage 808 (Attempt 0)

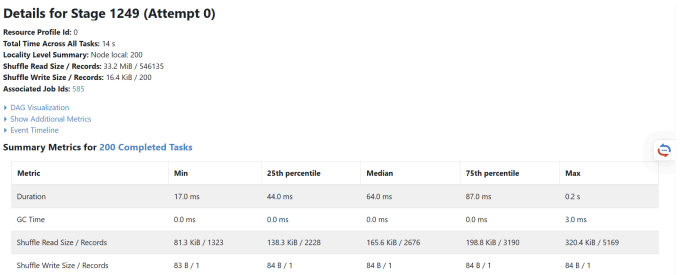
Resource Profile Id: 0
Total Time Across All Tasks: 17 s
Locality Level Summary: Node local: 200
Shuffle Read Size / Records: 11.0 MiB / 2764395
Shuffle Write Size / Records: 12.3 KiB / 200
Associated Job Ids: 387

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 200 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	33.0 ms	55.0 ms	73.0 ms	92.0 ms	0.3 s
GC Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	0.0 ms
Shuffle Read Size / Records	37.4 KiB / 8679	51 KiB / 12134	55.6 KiB / 13212	60.9 KiB / 14454	130.8 KiB / 119530
Shuffle Write Size / Records	59 B / 1	63 B / 1	63 B / 1	63 B / 1	63 B / 1

Fig. 6 — **Plain sort-merge join, AQE off, medium.** 200 tasks, 2,764,395 shuffle records. Records per task: min 8,679 / median 13,212 / **max 119,530** (max/median 9.0×, the bot partition); duration 33 / 73 / **300 ms** (4.1×). Bytes differ only 2.4× (55.6 vs 130.8 KiB) because the bot's rows are highly repetitive and compress well.



Details for Stage 1249 (Attempt 0)

Resource Profile Id: 0
Total Time Across All Tasks: 14 s
Locality Level Summary: Node local: 200
Shuffle Read Size / Records: 33.2 MiB / 546135
Shuffle Write Size / Records: 16.4 KiB / 200
Associated Job Ids: 585

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 200 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	17.0 ms	44.0 ms	64.0 ms	87.0 ms	0.2 s
GC Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	3.0 ms
Shuffle Read Size / Records	81.3 KiB / 1323	138.3 KiB / 2228	165.6 KiB / 2676	198.8 KiB / 3190	320.4 KiB / 5169
Shuffle Write Size / Records	83 B / 1	84 B / 1	84 B / 1	84 B / 1	84 B / 1

Fig. 7 — For comparison: **salted join, N = 64** (recorded on *small*). The hot key is spread over 64 partitions: records max/median 5,169 / 2,676 = 1.9×, duration 200 / 64 ms.

D. Quarantine reason combinations (medium)

dq_reasons	rows	dq_reasons	rows
[R1]	22,044	[R1, R3]	57
[R5]	11,153	[R1, R2]	56
[R3]	8,430	[R3, R5]	32
[R2]	5,542	[R2, R5]	29
[R1, R5]	86	[R2, R3]	16